



Código de Autenticação de Mensagem (MACs)

Prof. Dr. Iaçanã Ianiski Weber

Confiabilidade e Segurança de Software

98G08-4

Agradecimentos especiais ao Prof. Avelino Zorzo e aos Autores Christof Paar e Jan Pelzl pelo material.

Índice

- 1 Por que Precisamos de MACs
- 2 Princípios dos MACs
- 3 MACs a partir de Funções Hash
- 4 MACs a partir de Cifras de Bloco
- 5 Encriptação Autenticada (AE / AEAD)
- 6 Lições Aprendidas

Conteúdo deste Capítulo

Nesta aula você vai aprender:

- O **princípio** por trás dos MACs (*Message Authentication Codes*).
- As **propriedades de segurança** que os MACs oferecem, e a que eles **não** oferecem (não repúdio).
- Como MACs podem ser construídos a partir de **funções hash** (**HMAC** e **KMAC**).
- Como MACs podem ser construídos a partir de **cifras de bloco** (**CBC-MAC** e **CMAC**).
- Como combinar **confidencialidade + autenticação** em um único passo: **criptação autenticada** (**CCM**, **GCM**, **GMAC**).

- 1 Por que Precisamos de MACs
- 2 Princípios dos MACs
- 3 MACs a partir de Funções Hash
- 4 MACs a partir de Cifras de Bloco
- 5 Encriptação Autenticada (AE / AEAD)
- 6 Lições Aprendidas

Motivação: Integridade e Autenticação

O problema: Bob envia uma mensagem x a Alice por um canal inseguro. Como Alice tem certeza de que:

- a mensagem **não foi alterada** no caminho (**integridade**); e
- a mensagem realmente **veio de Bob** (**autenticação de origem**)?

Motivação: Integridade e Autenticação

O problema: Bob envia uma mensagem x a Alice por um canal inseguro. Como Alice tem certeza de que:

- a mensagem **não foi alterada** no caminho (**integridade**); e
- a mensagem realmente **veio de Bob** (**autenticação de origem**)?

Por que um hash sozinho não basta?

- Funções hash são **sem chave** (*keyless*): *qualquer um* pode calcular $h(x)$.
- Se Bob enviasse $(x, h(x))$, o atacante Oscar poderia trocar a mensagem por x' e simplesmente recalcular $h(x')$. Alice não perceberia.
- **Falta um segredo** que ligue a mensagem a quem a produziu.

Motivação: Integridade e Autenticação

O problema: Bob envia uma mensagem x a Alice por um canal inseguro. Como Alice tem certeza de que:

- a mensagem **não foi alterada** no caminho (**integridade**); e
- a mensagem realmente **veio de Bob** (**autenticação de origem**)?

Por que um hash sozinho não basta?

- Funções hash são **sem chave** (*keyless*): *qualquer um* pode calcular $h(x)$.
- Se Bob enviasse $(x, h(x))$, o atacante Oscar poderia trocar a mensagem por x' e simplesmente recalcular $h(x')$. Alice não perceberia.
- **Falta um segredo** que ligue a mensagem a quem a produziu.

A ideia do MAC: introduzir uma **chave secreta simétrica** k , compartilhada entre Bob e Alice, no cálculo do “checksum”. Só quem tem k consegue gerar uma *tag* válida.

Onde os MACs se Encaixam

Três ferramentas que produzem uma “etiqueta” a partir de uma mensagem:

	Função Hash	MAC	Assinatura Digital
Usa chave?	Não (keyless)	Simétrica	Assimétrica
Integridade	Sim*	Sim	Sim
Autenticação de origem	Não	Sim	Sim
Não repúdio	Não	Não	Sim
Velocidade	Rápida	Rápida	Lenta

- * O hash detecta alterações *acidentais*, mas não *maliciosas* se enviado sem proteção.
- **O MAC é o “meio-termo”**: oferece integridade e autenticação como a assinatura, mas com o desempenho da criptografia simétrica.
- **Preço a pagar**: como a chave é *compartilhada*, o MAC **não fornece não repúdio** (veremos por quê).

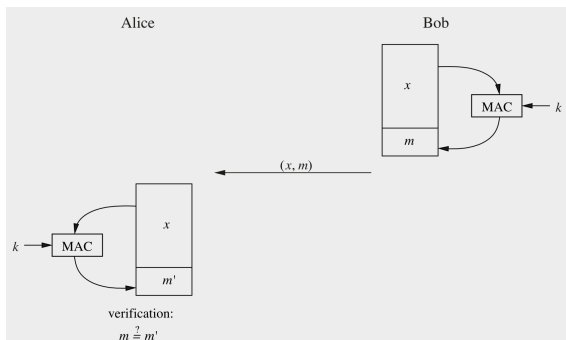
- 1 Por que Precisamos de MACs
- 2 Princípios dos MACs**
- 3 MACs a partir de Funções Hash
- 4 MACs a partir de Cifras de Bloco
- 5 Encriptação Autenticada (AE / AEAD)
- 6 Lições Aprendidas

Princípio dos MACs

- De forma semelhante às assinaturas digitais, MACs acrescentam uma *tag* de autenticação a uma mensagem.
- MACs utilizam uma **chave simétrica** k tanto para a **geração** quanto para a **verificação**.
- Cálculo de um MAC:

$$m = \text{MAC}_k(x)$$

é uma função da chave secreta k e da mensagem x .



O Protocolo Passo a Passo

Bob (remetente):

- 1 Calcula $m = \text{MAC}_k(x)$ com a chave secreta k .
- 2 Envia o par (x, m) para Alice.

Alice (receptora):

- 1 Recebe (x, m) .
- 2 Recalcula $m' = \text{MAC}_k(x)$ com a *mesma* chave k .
- 3 Verifica se $m' \stackrel{?}{=} m$.

Interpretação do resultado:

- $m' = m \Rightarrow$ mensagem **íntegra** e **autêntica** (só quem tem k poderia ter gerado m).
- $m' \neq m \Rightarrow$ a mensagem x e/ou a tag m foram **alteradas** em trânsito.

Premissa fundamental: se x for alterada em trânsito, o MAC recomputado dará um valor *diferente*. Oscar não consegue forjar m sem conhecer k .

Propriedades dos Códigos de Autenticação de Mensagens

- 1 **Checksum criptográfico:** um MAC gera uma *tag* de autenticação criptograficamente segura para uma dada mensagem.
- 2 **Simétrico:** MACs são baseados em chaves simétricas secretas. As partes que geram e verificam o MAC devem *compartilhar* uma chave secreta.
- 3 **Tamanho arbitrário de mensagem:** MACs aceitam mensagens de comprimento arbitrário.
- 4 **Comprimento de saída fixo:** MACs geram *tags* de tamanho fixo (independente do tamanho da entrada).
- 5 **Integridade da mensagem:** qualquer modificação na mensagem em trânsito será detectada pelo receptor.
- 6 **Autenticação da mensagem:** a parte que recebe é assegurada quanto à origem legítima da mensagem.
- 7 **Ausência de não repúdio:** como MACs são baseados em princípios simétricos, eles **não** fornecem não repúdio.

Por que MACs Não Fornecem Não Repúdio?

- Alice e Bob **compartilham a mesma chave** k .
- Logo, *ambos* são capazes de gerar qualquer *tag* válida $m = \text{MAC}_k(x)$.

Por que MACs Não Fornecem Não Repúdio?

- Alice e Bob **compartilham a mesma chave** k .
- Logo, *ambos* são capazes de gerar qualquer *tag* válida $m = \text{MAC}_k(x)$.
- Diante de um juiz, é **impossível provar** quem dos dois gerou a mensagem:
 - Bob pode alegar que foi Alice quem forjou a mensagem (e vice-versa).
 - A chave simétrica está ligada a *duas* partes, não a *uma* pessoa (ao contrário da chave privada assimétrica).

Por que MACs Não Fornecem Não Repúdio?

- Alice e Bob **compartilham a mesma chave** k .
- Logo, *ambos* são capazes de gerar qualquer *tag* válida $m = \text{MAC}_k(x)$.
- Diante de um juiz, é **impossível provar** quem dos dois gerou a mensagem:
 - Bob pode alegar que foi Alice quem forjou a mensagem (e vice-versa).
 - A chave simétrica está ligada a *duas* partes, não a *uma* pessoa (ao contrário da chave privada assimétrica).
- **Conclusão:** MACs não protegem em cenários onde uma das partes pode ser desonesta (ex.: o caso da compra/venda de carro visto em assinaturas digitais).

Quando o não repúdio importa $\Rightarrow$ use assinatura digital. Quando importa velocidade entre partes que confiam mutuamente $\Rightarrow$ use MAC.

Quão Longa Deve Ser a *Tag* de um MAC?

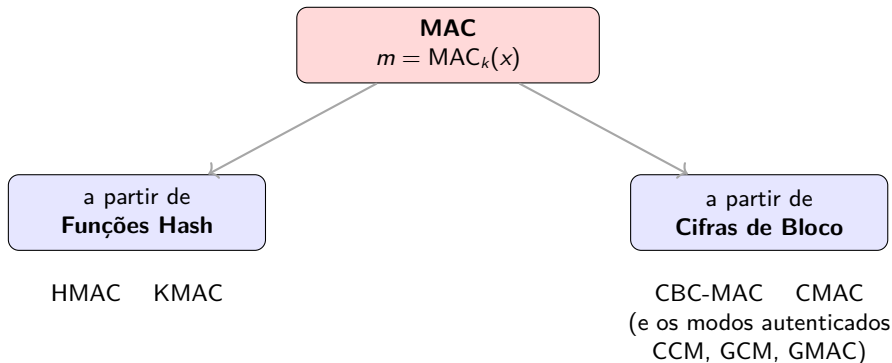
Diferente das funções hash! Em um hash, precisamos de saída ≥ 256 bits para resistir ao *birthday attack*. Em um MAC, **tags de 128 bits costumam bastar**. Por quê?

- No *birthday attack* contra um hash, o atacante trabalha **offline**: gera livremente milhões de pares e procura uma colisão $\approx 2^{n/2}$.
- Contra um MAC, o atacante **não** consegue calcular $\text{MAC}_k(\cdot)$ sozinho, pois lhe *falta a chave k* . Para testar um palpite de *tag*, ele precisa enviá-lo e ver se é aceito (ataque **online**).
- Forjar uma *tag* de n bits por tentativa-e-erro exige $\approx 2^n$ tentativas *online* (não $2^{n/2}$). Com $n = 128$, isso é proibitivo.

Falsificação existencial (*existential forgery*): o objetivo do atacante é produzir um par válido (x, m) que ele nunca viu antes, sem necessariamente escolher qual mensagem. Um bom MAC torna isso inviável.

Duas Famílias de Construção

Na prática, MACs são construídos de duas formas essencialmente diferentes:



- Estudaremos as duas famílias nas próximas seções.

- 1 Por que Precisamos de MACs
- 2 Princípios dos MACs
- 3 MACs a partir de Funções Hash**
- 4 MACs a partir de Cifras de Bloco
- 5 Encriptação Autenticada (AE / AEAD)
- 6 Lições Aprendidas

A Ideia Básica e as Tentativas Ingênuas

Podemos construir um MAC usando uma função hash criptográfica (SHA-2, SHA-3).

Ideia central: a chave é “*hasheada*” em conjunto com a mensagem.

Duas construções óbvias surgem de imediato:

- **MAC com prefixo secreto:** $m = \text{MAC}_k(x) = h(k \parallel x)$
- **MAC com sufixo secreto:** $m = \text{MAC}_k(x) = h(x \parallel k)$

($\parallel$ denota **concatenação**.)

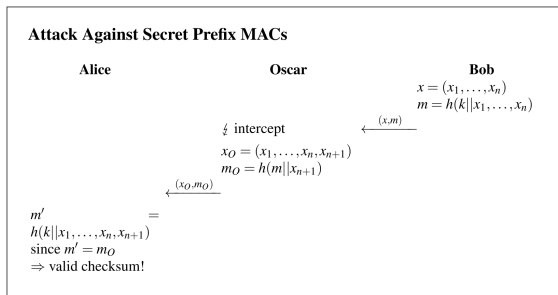
Cuidado! Apesar de parecerem seguras (graças à mão-única e às boas propriedades de “embaralhamento” do hash), **ambas têm fraquezas**. Vamos demonstrá-las, pois são elas que motivam o HMAC.

Ataque ao Prefixo Secreto: *Length Extension*

Considere $m = h(k \parallel x)$ com um hash **Merkle–Damgård** (caso de SHA-1 e SHA-2).

- A mensagem é $x = (x_1, \dots, x_n)$ e Bob calcula $m = h(k \parallel x_1, \dots, x_n)$.
- **Problema:** em Merkle–Damgård, o hash m é *exatamente* o estado interno após a mensagem. O atacante pode **continuar** o cálculo, sem k , e forjar a *tag* da mensagem **estendida** $x' = (x_1, \dots, x_n, x_{n+1})$:

$$m_O = h(m \parallel x_{n+1}) = h(k \parallel x_1, \dots, x_n, x_{n+1})$$



O bloco x_{n+1} poderia ser, p. ex., um **apêndice malicioso** a um contrato eletrônico.

Por que o SHA-3 Não Sofre Esse Ataque

- O *length extension attack* explora a estrutura **Merkle–Damgård**, onde a saída é o estado interno completo.
- O **SHA-3** usa a **construção esponja**: os c bits de *capacidade* do estado **nunca são emitidos** na saída.
- Sem acesso a esses bits ocultos, o atacante **não consegue continuar** o cálculo a partir da *tag*.
- Por isso, MACs baseados em SHA-3 (como o **KMAC**, que veremos adiante) dispensam o “embrulho” que o HMAC precisa fazer sobre SHA-2.

Funções hash modernas também incluem um *indicador de comprimento* no preenchimento para mitigar o ataque, mas mesmo assim a fraqueza estrutural do prefixo secreto continua preocupante.

Ataque ao Sufixo Secreto: Colisões

Considere agora $m = h(x \| k)$. Aqui surge *outra* fraqueza.

- Suponha que Oscar consiga construir uma **colisão** no hash, isto é, achar x e x_0 tais que

$$h(x) = h(x_0), \quad x \neq x_0.$$

- As duas mensagens podem ser, por exemplo, duas versões de um contrato que diferem no valor do pagamento.
- Se Bob assina x com $m = h(x \| k)$, então **m também é válida para x_0** :

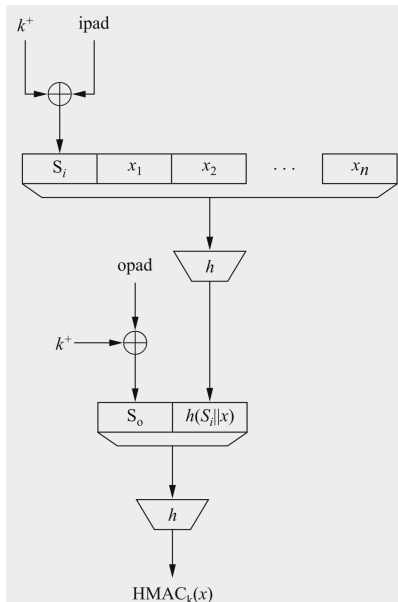
$$m = h(x \| k) = h(x_0 \| k).$$

E o paradoxo do aniversário ataca aqui: mesmo com SHA-256 (256 bits) e chave de 256 bits, esperaríamos 2^{256} de segurança, mas uma colisão custa apenas $\approx \sqrt{2^{256}} = 2^{128}$.

Conclusão: o sufixo secreto é inseguro se uma colisão no hash subjacente puder ser encontrada. Precisamos de algo melhor: o **HMAC**.

HMAC: Visão Geral

- Proposto por **Mihir Bellare, Ran Canetti e Hugo Krawczyk** em 1996.
- Não sofre as fraquezas do prefixo nem do sufixo secreto.
- Consiste em um **hash interno** (*inner*) e um **hash externo** (*outer*), o que explica o nome *hash-based* MAC.
- Amplamente usado na prática: **TLS** e **IPsec**.
- Possui **prova de segurança**: continua seguro mesmo que se encontre uma colisão ou segunda pré-imagem no hash subjacente.



HMAC: A Construção em Detalhe

Passo 1: expansão da chave. Anexa-se bytes zero à chave k até obter k^+ com b bits, onde b é a **largura do bloco de entrada** do hash (ex.: $b = 512$ para SHA-256).

Passo 2: dois *pads* fixos (padrões de bits repetidos, de b bits):

$$\text{ipad} = 00110110, \dots \quad \text{opad} = 01011100, \dots$$

Passo 3 (hash interno): processa $(k^+ \oplus \text{ipad})$ seguido da mensagem x .

Passo 4 (hash externo): processa $(k^+ \oplus \text{opad})$ seguido da saída do hash interno.

$$\text{HMAC}_k(x) = h\left((k^+ \oplus \text{opad}) \parallel h((k^+ \oplus \text{ipad}) \parallel x)\right)$$

Os valores exatos de ipad/opad não importam para a segurança; foram escolhidos apenas para que a chave mascarada *difira em muitos bits* entre os dois hashes.

HMAC: Eficiência e Segurança

Eficiência: o custo extra é mínimo.

- A mensagem x (que pode ser longa) é processada **uma única vez**, no hash interno.
- O hash externo processa apenas **dois blocos**: a chave mascarada e a saída do hash interno.
- A “interface” entre os dois hashes não é problema: o hash externo recebe uma entrada curta (saída interna de l bits), e o pré-processamento do hash ajusta ao tamanho de bloco b .

Segurança: a grande vantagem.

- Existe uma **prova de segurança** formal.
- Pode-se mostrar que o HMAC permanece seguro **mesmo que** uma colisão ou segunda pré-imagem seja encontrada na função hash subjacente.
- É por isso que o HMAC continua sendo usado com SHA-2 com tranquilidade, mesmo após os problemas históricos de colisão em hashes mais antigos.

KMAC: o MAC Nativo do SHA-3

Motivação: o HMAC “embrulha” o hash duas vezes para contornar o *length extension*. Com o **SHA-3 (esponja)**, esse problema *nem existe*. Surge então um MAC mais direto.

- **KMAC** = *KECCAK Message Authentication Code*.
- Padronizado pelo **NIST** em **2016**, na publicação **SP 800-185** (“*SHA-3 Derived Functions*”).
- É um **MAC** e **também uma PRF** (função pseudoaleatória) baseado em KECCAK.
- É **nativamente chaveado**: a chave entra diretamente na entrada da esponja, **sem** a construção aninhada do HMAC.
- Junto com KMAC, a SP 800-185 define **TupleHash** e **ParallelHash** (não são MACs; são outros usos do KECCAK).

Cadeia de derivação:

$\text{KECCAK-}f \rightarrow \text{SHA-3 / SHAKE (FIPS 202)} \rightarrow \text{cSHAKE} \rightarrow \text{KMAC}$.

cSHAKE: o “Tijolo” do KMAC

cSHAKE (*customizable SHAKE*) é a versão *customizável* da XOF SHAKE. Recebe quatro parâmetros:

$$\text{cSHAKE}(X, L, N, S)$$

- X = entrada principal; L = comprimento de saída desejado (bits);
- N = *function-name string* (reservada ao NIST, faz a separação por *nome de função*);
- S = *customization string* (escolhida pelo usuário, faz a separação por *uso*).
- Quando N e S são *ambas vazias*, cSHAKE **degenera no SHAKE comum**:

$$\text{cSHAKE128}(X, L, "", "") = \text{SHAKE128}(X, L).$$

- Qualquer customização não vazia produz uma saída **separada por domínio** do SHAKE puro. Na prática, é *outra função*, sem colisões entre usos diferentes.

KMAC128 e KMAC256: Parâmetros

A permutação é sempre **KECCAK-f[1600]**, com largura $b = 1600 = r + c$ (*rate* + *capacity*). A capacidade c define o nível de segurança:

Variante	Seg.	b	c (cap.)	r (rate)	r em bytes
KMAC128 (cSHAKE128)	128 bits	1600	256	1344	168
KMAC256 (cSHAKE256)	256 bits	1600	512	1088	136

- Confira: $168 \times 8 = 1344 = 1600 - 256$ e $136 \times 8 = 1088 = 1600 - 512$.
- **Maior capacidade** $c \Rightarrow$ **maior segurança**, mas *rate* menor (menos bits absorvidos por chamada $\Rightarrow$ um pouco mais lento).
- O *rate em bytes* (168 / 136) aparecerá como parâmetro do bytepad na construção.

KMAC: A Construção

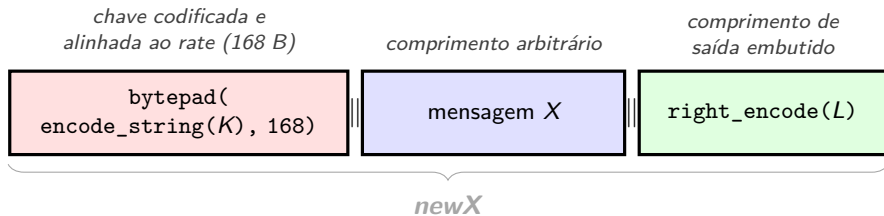
Assinatura: $\text{KMAC}(K, X, L, S)$

- K = chave (qualquer comprimento); X = mensagem; L = comprimento de saída em bits; S = string de customização opcional.

KMAC128 (*rate* = 168 bytes):

- 1 $\text{newX} = \text{bytepad}(\text{encode_string}(K), 168) \parallel X \parallel \text{right_encode}(L)$
- 2 **retorna** $\text{cSHAKE128}(\text{newX}, L, \text{"KMAC"}, S)$

KMAC256 (*rate* = 136 bytes): idem, trocando 168 $\rightarrow$ 136 e $\text{cSHAKE128} \rightarrow \text{cSHAKE256}$.



newX é então absorvido por $\text{cSHAKE128}(\text{newX}, L, \text{"KMAC"}, S)$, que produz a *tag* de L

KMAC: As Funções de Codificação (e por que importam)

Tudo gira em torno de **enquadramento injetivo e auto-delimitado**: cada campo carrega seu próprio comprimento, de modo que as fronteiras (chave / mensagem / comprimento) são recuperáveis sem ambiguidade.

- `left_encode(x)`: codifica o inteiro x *prefixando* o número de bytes usados (parseável *do início*).
- `right_encode(x)`: o mesmo, mas com a contagem *no final* (parseável *do fim*).
- `encode_string(S) = left_encode(len(S)) || S`: prefixa o comprimento à string.
- `bytepad(X, w)`: prefixa `left_encode(w)` e completa com bytes zero até o total ser múltiplo de w bytes (alinha a chave ao *rate* da esponja).

Por quê? Sem “etiquetar” o comprimento de cada campo, um atacante poderia mover bytes entre a chave e a mensagem e produzir o *mesmo* $newX$ a partir de $(K', X') \neq (K, X)$. As codificações eliminam essa **confusão de fronteiras**. É isso que transforma o ingênuo (e inseguro) $h(K||X)$ em um MAC sólido.

left_encode e right_encode em Detalhe

Ambas codificam um inteiro x junto com *quantos bytes* ele ocupa. Seja n o menor número de bytes que representa x em base 256 (*big-endian*):

$$\text{left_encode}(x) = \underbrace{[n]}_{1 \text{ byte}} \parallel [x_1] \parallel \cdots \parallel [x_n]$$

$$\text{right_encode}(x) = [x_1] \parallel \cdots \parallel [x_n] \parallel \underbrace{[n]}_{1 \text{ byte}}$$

A diferença é só a posição do byte de contagem n : na frente (*left*, lido do início) ou no fim (*right*, lido do fim).

x	$\text{left_encode}(x)$
0	01 00
1	01 01
168	01 A8
256	02 01 00

x	$\text{right_encode}(x)$
0	00 01
1	01 01
168	A8 01
256	01 00 02

Bytes em hexadecimal. Ex.: 256 = 0x0100 ocupa $n = 2$ bytes; o byte de contagem é 02.

encode_string e bytepad

Com left_encode pronta, as outras duas são imediatas.

- `encode_string(S)` prefixa à string S o seu **comprimento em bits**:

$$\text{encode_string}(S) = \text{left_encode}(\text{bitlen}(S)) \parallel S$$

(Para S vazia: `encode_string('')` = `left_encode(0)` = 01 00.)

- `bytepad(X, w)` prepõe `left_encode(w)` e completa com bytes 00 até o total ser **múltiplo de w bytes**:

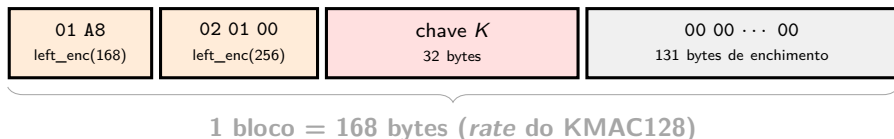
$$\text{bytepad}(X, w) = \text{left_encode}(w) \parallel X \parallel \underbrace{00 \dots 00}_{\text{enchimento}}$$

Papel de cada uma: `encode_string` torna a chave *auto-delimitada* (sabe-se onde ela começa e termina); `bytepad` a *alinha ao rate* da esponja, garantindo que a chave ocupe um número inteiro de blocos absorvidos.

Exemplo: Montando o Bloco da Chave no KMAC128

Seja uma chave K de **256 bits** (32 bytes). O primeiro bloco absorvido é $\text{bytepad}(\text{encode_string}(K), 168)$. Vamos montá-lo:

- 1 $\text{encode_string}(K) = \text{left_encode}(256) \parallel K = \underbrace{02\ 01\ 00}_{3\text{ bytes}} \parallel \underbrace{K}_{32\text{ bytes}}$ (35 bytes).
- 2 $\text{bytepad}(\cdot, 168)$ prepõe $\text{left_encode}(168) = 01\ A8$ (2 bytes) $\Rightarrow$ 37 bytes.
- 3 Completa com $168 - 37 = 131$ bytes $00 \Rightarrow$ exatamente **168 bytes** (= o *rate*).



Para o **KMAC256** tudo é igual, trocando o *rate* para **136 bytes** ($\text{left_encode}(136) = 01\ 88$).

KMAC: Por que Codificar o Comprimento de Saída L ?

A propriedade que mais distingue o KMAC: L é codificado *dentro da entrada* (via `right_encode(L)`).

- Consequência: **mudar o comprimento de saída solicitado gera uma saída totalmente nova e não relacionada.**
- **Contraste com SHAKE/cSHAKE (XOF puro):** ali, a saída de 128 bits é simplesmente o *prefixo* da saída de 256 bits, de modo que as duas são trivialmente relacionadas.
- Ligando L à entrada absorvida, o KMAC garante que a *tag* de 128 bits **não** seja prefixo da de 256 bits para a mesma (K, X) , evitando problemas de *related-output* e de extensão.
- Usa-se `right_encode` (e não `left_encode`) porque L fica *no final* de *newX* e precisa ser lido a partir do fim, sem conhecer `len(X)` de antemão.

KMAC: Separação de Domínio e Variante XOF

Separação de domínio em duas camadas:

- N = “KMAC” (fixo): separa o KMAC do SHAKE puro e de outras funções NIST (TupleHash usa N = “TupleHash”, etc.). *O usuário não deve inventar o próprio N .*
- S (customização): separa diferentes *usos* da mesma função, pois customizações distintas $\Rightarrow$ saídas não relacionadas. Não há “valores fracos” de N ou S .

Variante XOF (KMACXOF128 / KMACXOF256):

- Idêntica ao KMAC, exceto que anexa `right_encode(0)` em vez de `right_encode(L)`.
- Produz uma **saída de comprimento arbitrário / streamável** (a saída curta é prefixo da longa, como no cSHAKE).
- Útil quando se quer um fluxo de bits pseudoaleatórios chaveado de tamanho indefinido.

Detalhe sutil: o modo XOF é acionado por *anexar* `right_encode(0)`, e não por passar $L = 0$ na chamada; o L continua dizendo quantos bits serão emitidos.

KMAC vs. HMAC

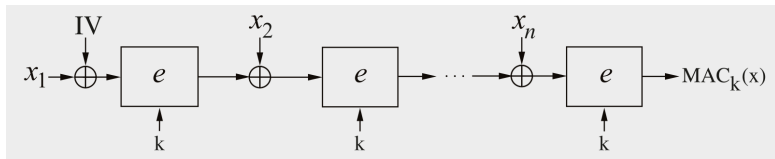
	HMAC	KMAC
Base	SHA-1 / SHA-2 (Merkle–Damgård)	SHA-3 / KECCAK (esponja)
Chaveamento	aninhado (<i>inner+outer</i>)	nativo, passada única
<i>Length extension</i>	contornado pelo aninhamento	imune por construção
Comprimento de saída	fixo (truncável)	variável, ligado à entrada
Customização/domínio	não nativa	embutida (<i>N, S</i>)
Maturidade/adoção	enorme (TLS, IPsec)	crescente (libs, KEMs)

- **Segurança:** KMAC128 → 128 bits; KMAC256 → 256 bits. A chave deve ter pelo menos o nível de segurança desejado.
- **Recomendações de saída:** não usar *tags* com $L < 32$ bits; usar $L < 64$ bits só após análise de risco.
- **Sobre desempenho:** a passada única é uma vantagem *estrutural*; mas “mais rápido que HMAC” **depende do hardware** (aceleração de KECCAK vs. SHA-2).

- 1 Por que Precisamos de MACs
- 2 Princípios dos MACs
- 3 MACs a partir de Funções Hash
- 4 MACs a partir de Cifras de Bloco**
- 5 Encriptação Autenticada (AE / AEAD)
- 6 Lições Aprendidas

CBC-MAC: Visão Geral

- MAC construído a partir de uma cifra de bloco (ex.: AES).
- Por muito tempo, foi a abordagem **mais popular**: usa o modo **CBC** (*Cipher Block Chaining*).
- A *tag* é a **saída do último bloco**: $m = \text{MAC}_k(x) = y_n$.
- Como tudo depende em cadeia, m é função de *todos* os blocos $x_1, \dots, x_n$.



CBC-MAC: Geração e Verificação

Geração do MAC

- Dividir a mensagem x em blocos x_i .
- Primeira iteração: $y_1 = e_k(x_1 \oplus IV)$.
- Demais blocos: $y_i = e_k(x_i \oplus y_{i-1})$.
- A *tag* é o último bloco:
 $m = \text{MAC}_k(x) = y_n$.
- **Diferenças em relação ao CBC de cifragem:** (i) não se transmitem os valores intermediários, só o último y_n ; (ii) o receptor **não** precisa decifrar, pois usa e sempre no modo de cifragem.
- **Comprimento da *tag*** = tamanho de bloco da cifra (AES $\Rightarrow$ 128 bits).

Verificação do MAC

- Receptor *recomputa* o CBC-MAC obtendo m' .
- Se $m' = m$: mensagem íntegra/correta.
- Se $m' \neq m$: a mensagem e/ou a *tag* foram alteradas em trânsito.

CBC-MAC: Cuidados e Fraquezas

- **O IV deve ser fixo** (ex.: zero).
 - Caso contrário, um atacante poderia alterar o primeiro bloco x_1 *junto* com uma mudança correspondente no IV, resultando no *mesmo* MAC.
- **Fraqueza de segurança (falsificação existencial)**: dados os MACs de mensagens autenticadas *individualmente*, um atacante pode construir um MAC válido para uma **nova** mensagem formada pela *concatenação* delas.
- Apesar disso, o CBC-MAC foi (e ainda é) usado na prática, inclusive padronizado com DES para transações financeiras (FIPS 113, expirado em 2008).

CBC-MAC: Cuidados e Fraquezas

- **O IV deve ser fixo** (ex.: zero).
 - Caso contrário, um atacante poderia alterar o primeiro bloco x_1 *junto* com uma mudança correspondente no IV, resultando no *mesmo* MAC.
- **Fraqueza de segurança (falsificação existencial)**: dados os MACs de mensagens autenticadas *individualmente*, um atacante pode construir um MAC válido para uma **nova** mensagem formada pela *concatenação* delas.
- Apesar disso, o CBC-MAC foi (e ainda é) usado na prática, inclusive padronizado com DES para transações financeiras (FIPS 113, expirado em 2008).
- **Solução**: uma pequena modificação corrige essas fraquezas $\Rightarrow$ **CMAC**.

CMAC: o CBC-MAC Corrigido

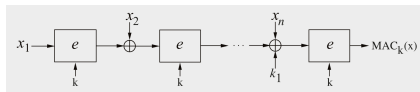
CMAC (*Cipher-based MAC*) é próximo do CBC-MAC, mas remove suas fraquezas. **Duas diferenças:**

- 1 Sem *IV* (ou, equivalentemente, *IV* fixo em zero).
- 2 Antes de cifrar o **último** bloco x_n , soma-se (XOR) uma **subchave** k_1 :

$$k_1 = e_k(0)$$

(versão simplificada: cifra-se a string de zeros).

No padrão NIST, k_1 ainda passa por rotações e uma máscara fixa, conforme o caso.



- O usuário fornece **apenas** k ; a subchave k_1 é *derivada* dela.
- CMAC deriva de OMAC (Iwata–Kurosawa, 2003) e é padronizado pelo NIST (SP 800-38B). Na prática, usa-se com AES.

- 1 Por que Precisamos de MACs
- 2 Princípios dos MACs
- 3 MACs a partir de Funções Hash
- 4 MACs a partir de Cifras de Bloco
- 5 **Encriptação Autenticada (AE / AEAD)**
- 6 Lições Aprendidas

Confidencialidade e Autenticação no Mesmo Passo

Necessidade prática: muitas aplicações querem, ao mesmo tempo,

- **confidencialidade** (via cifragem) e
- **autenticação + integridade** (via MAC).

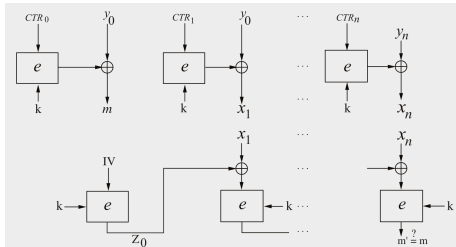
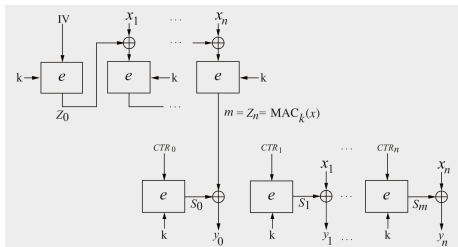
Poderíamos combinar um modo de cifragem (Cap. 5) *com* CBC-MAC/CMAC. Mas é mais atraente ter uma primitiva que faça **cifragem e MAC em uma só passada: encriptação autenticada (AE)**.

Modo	Cifragem	Autenticação	Comentário
CBC-MAC		X	deficiências de segurança
CMAC		X	
CCM	X	X	encriptação autenticada
GCM	X	X	encriptação autenticada
GMAC		X	variante do GCM, só MAC

Tabela 13.1 do livro. CMAC, CCM, GCM e GMAC foram padronizados pelo NIST entre 2004–2007 (série SP 800-38).

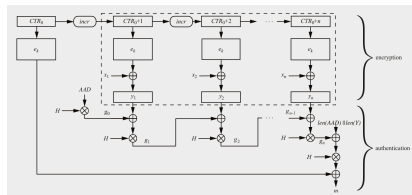
CCM: Counter with CBC-MAC

- Combina o modo **contador (CTR)** para cifragem com o **CBC-MAC** para autenticação, usando a *mesma* chave k .
- Opera no esquema ***authenticate-then-encrypt***: primeiro calcula o MAC, depois cifra a *tag* m junto com a mensagem.
- A *tag* m **não** trafega em claro: ela é cifrada e vira o primeiro bloco de saída y_0 .
- O IV deve ser um **nonce** (valor usado uma única vez) e distinto dos valores de contador CTR_i .
- Muito usado: **WPA2 (Wi-Fi)**, **Bluetooth LE**, **IPsec**.

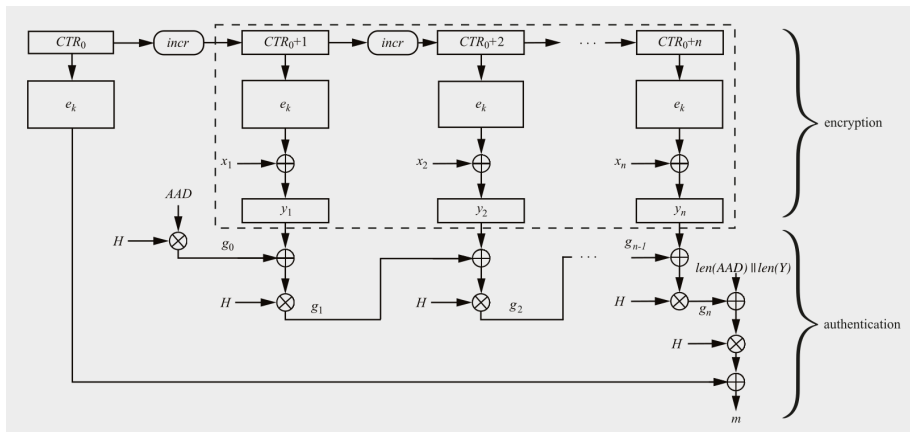


GCM: *Galois Counter Mode*

- Combina cifragem (modo CTR) e MAC. Muito usado em **TLS** e **IPsec**.
- Esquema ***encrypt-then-authenticate***: cifra primeiro, calcula o MAC sobre os *ciphertexts*.
- Fornece **AEAD** (*Authenticated Encryption with Associated Data*): parte da mensagem (ex.: cabeçalhos) *não* é cifrada, mas é autenticada.
- Autenticação via **multiplicações no corpo de Galois $GF(2^{128})$** , com subchave $H = e_k(0)$.
- Cada ciphertext y_i é combinado por XOR com o resultado anterior e multiplicado por H (encadeamento).



GCM: Diagrama Completo (Fig. 13.7)



GCM: Detalhes e GMAC

Sobre o GCM:

- As multiplicações são em $GF(2^{128})$ com $P(x) = x^{128} + x^7 + x^2 + x + 1 \Rightarrow$ exige cifra de **bloco de 128 bits** (ex.: AES).
- Apenas *uma* multiplicação por bloco $\Rightarrow$ custo de autenticação muito baixo.
- Por ligar os dados associados (AAD) ao ciphertext, detecta ataques de “recombinação” de cabeçalhos.

GMAC, a variante “só MAC”:

- É o GCM **sem cifragem**: computa *apenas* o código de autenticação.
- Para aplicações que precisam de integridade/autenticidade, mas *não* de confidencialidade.
- Altamente **paralelizável** $\Rightarrow$ eficiente em hardware: throughputs de **100 Gbit/s ou mais**.
- Padronizado e usado no **IPsec** (ESP e *Authentication Header*).

- 1 Por que Precisamos de MACs
- 2 Princípios dos MACs
- 3 MACs a partir de Funções Hash
- 4 MACs a partir de Cifras de Bloco
- 5 Encriptação Autenticada (AE / AEAD)
- 6 Lições Aprendidas**

Lições Aprendidas

- MACs fornecem **dois serviços de segurança**: *integridade e autenticação* de mensagem, usando técnicas **simétricas**. São amplamente usados em protocolos práticos.
- Esses serviços também são oferecidos por assinaturas digitais, mas MACs são **muito mais rápidos** (algoritmos simétricos).
- MACs **não fornecem não repúdio** (a chave é compartilhada).
- MACs de relevância prática baseiam-se em **cifras de bloco** ou em **funções hash**.
- **HMAC** é um MAC popular baseado em hash, com prova de segurança; muito usado (TLS, IPsec). **KMAC** é o equivalente nativo do SHA-3: chaveado por construção, com saída de tamanho variável e separação de domínio embutidas.
- **Encriptação autenticada** (CCM, GCM) entrega confidencialidade e autenticação em um passo; **GMAC** é a variante “só autenticação” do GCM.